

One Profile to Rule Them All: A Case Study in Persistent Multi-Tier AI Companionship

Martin Grobbelaar
Independent Researcher

July 2026

Abstract

We present a case study of a single-profile, multi-tier AI agent architecture deployed over four months of sustained daily use, with a central claim: the architecture’s primary contribution is **relationship continuity**—the frictionless human-AI interaction that comes from persistent identity across sessions, tasks, and cognitive tiers. Unlike the two dominant paradigms—stateless single-session agents and profile-switching between isolated personas—our approach accumulates one persistent identity across three operational tiers: an Orchestrator for routine work, a Worker for autonomous multi-step tasks, and an Analyst for deep reasoning. All three share a unified skill library, memory store, and aesthetic substrate.

We evaluate this architecture through a controlled N=5 benchmark (Phonebook task) comparing a blank agent (C1), a skill-configured agent (C2), and the fully evolved profile (C3). All conditions produced working applications across 15 trials, but C3 exhibited the tightest output variance (41–49 files vs. C1’s 26–72), spontaneous deployment of learned aesthetic preferences (Catppuccin dark theme, unprompted), and a consistent downward trend in token consumption as memory accumulated. C2 demonstrated stochastic delegation emergence (~50% activation)—skills enabled but did not guarantee advanced patterns.

These benchmark results are the *supporting evidence*, not the headline. The headline is qualitative: over four months, the profile developed behaviors never explicitly programmed—personality emergence, no-re-explain continuity, the three-attempt rule, skill provenance awareness, and ethical transparency that increased rather than reduced trust. We identify the C3 advantage not as a binary capability difference but as a gradient of reliability, taste, and procedural transfer acquired through persistent identity. We extract four generalizable design principles and discuss implications for long-lived AI assistants, the challenge of benchmarking systems whose value proposition is relationship quality rather than raw capability, and aesthetic learning as a signal of genuine preference formation.

1 Introduction

LLM agents are typically stateless: a model is prompted, tools are invoked, and the session ends with no persistent trace. Multi-agent architectures compound this by spawning ephemeral sub-agents that dissolve on completion.

One common response is **profile switching**: users create multiple personas—one for coding, one for writing, one for research—each with its own system prompt, memory, and tools. The idea is compartmentalization. The cost is fragmentation. Preferences learned in one context never transfer to another. The user becomes the only bridge between profiles, carrying the cognitive load of remembering which persona knows what. The system scales by adding more amnesiacs.

This paper offers an alternative: a **single-profile, multi-tier architecture**. Instead of switching between personas, one identity operates at different cognitive depths. An Orchestrator tier handles routine work and relationship continuity. A Worker tier handles autonomous multi-step workflows. An Analyst tier handles deep reasoning and code audits. All three share the same memory store, skill library, and aesthetic conventions. Nothing is siloed. Everything accumulates.

We report on a four-month deployment of this architecture on the Hermes Agent framework.¹ The profile operates three back-ends under one identity: the **Orchestrator** (DeepSeek V4 Flash) for routine tasks; a **Worker** (Hermes 3 8B, locally deployed via Ollama, cost-free) for autonomous self-directed work and delegation; and an **Analyst** (DeepSeek V4 Pro) for deep analysis and reasoning. All three share 158 authored skills, a persistent memory store, and a single configuration surface.

Our central question is not “Can a configured agent outperform a blank one?” —that would be trivial. Our question is: *Does persistent identity produce a qualitatively different interaction experience, and can that difference be measured?* To answer, we designed a controlled benchmark in which the same task—building a full-stack phonebook app (Angular + FastAPI + PostgreSQL)—is executed under three conditions: a blank agent (C1), a skill-configured agent (C2), and the fully evolved profile (C3). N=5 trials per condition, 15 runs total.

But we treat the benchmark as corroborating evidence, not the primary contribution. Over four months of daily use, the profile accumulated behaviors that no benchmark alone could capture: the user practically never had to re-introduce themselves or re-explain their projects; a consistent personality emerged that the user described as “recognisable” across sessions; the agent developed a three-attempt rule balancing persistence with pragmatism; it could trace its own competence to specific shared projects rather than generic training data; and explicit ethical boundaries increased trust rather than reducing it. These qualitative observations constitute the paper’s primary contribution. The benchmark data shows that these behaviors *do* translate into measurable improvements in consistency, efficiency, and aesthetic refinement—but the headline is the relationship, not the metrics.

This paper makes four contributions:

1. A critique of the profile-switching paradigm: compartmentalization trades continuity for focus, and the user pays the integration tax.
2. The single-profile multi-tier architecture: one identity, three operational depths, shared memory and skills.
3. A concrete case study with controlled benchmark data (N=5, 15 runs) and four months of qualitative observations demonstrating relationship continuity as a measurable phenomenon.
4. Four generalizable design principles—identity unity, tiered cognition, procedural memory, aesthetic accumulation—extracted from the implementation and independent of any framework.

2 Background & Related Work

2.1 The Profile-Switching Paradigm

Profile switching—maintaining separate personas for different domains—is a pragmatic response to a real problem: a single system prompt cannot simultaneously encode the conventions of software engineering, academic writing, and creative work without becoming contradictory. The hidden cost is **relationship fragmentation**. Each profile starts fresh. Preferences about style, tooling, and

¹<https://hermes-agent.nousresearch.com>

conventions must be re-established in each silo. The user carries the memory of which profile knows what. And profiles cannot learn from each other: a bug-workaround discovered in one stays in that silo.

2.2 Multi-Agent Systems

Multi-agent systems (MAS) have a long history [1], from classical distributed problem-solving to modern LLM-based orchestration [2, 3]. Frameworks such as AutoGen [4], CrewAI, and LangGraph let developers compose agents with distinct roles and handoff protocols. But these systems treat each agent as a fresh deployment; any “personality” must be re-injected via system prompts at startup. Our work differs: a single persistent profile accumulates state across months, with memory and skills compounding organically.

2.3 Persona and Identity in AI

Research on persona-consistent LLMs focuses on within-session coherence [5]. Long-term memory approaches augment LLMs with RAG over past interactions [6], but these are reactive—they retrieve relevant history without shaping default behavior. Our architecture treats identity as *constitutive*: memory, skills, and preferences become the default operating state, not just a retrievable archive.

2.4 Skill Libraries and Procedural Transfer

Skill libraries for LLM agents—reusable chunks of procedural knowledge—have been explored in Voyager [7] and the Hermes skill system. Prior work has not examined how a skill library *compounds* with persistent memory: do skills alone suffice, or does their effectiveness depend on the agent knowing *when* to load them?

2.5 Cognitive Architectures and Tool-Use

Cognitive architectures such as ACT-R [8] and Soar distinguish procedural from declarative memory and manage impasses through automatic sub-goaling. Our architecture independently converged on four parallel patterns:

1. **Memory hierarchies:** ACT-R’s long-term/working-memory distinction mirrors the layering of durable memory, skills, and holographic associative store in our system.
2. **Impasse-driven escalation:** Soar’s mechanism of detecting when no rule applies and spawning a sub-goal mirrors the three-attempt delegation rule—when the agent cannot resolve a task directly, it escalates to a specialist.
3. **Chunking as skill creation:** Soar compiles successful problem-solving into new rules; our system captures successful interaction patterns as authored skill files.
4. **Activation-based retrieval:** ACT-R’s spreading activation over declarative memory parallels holographic memory’s trust-scoring mechanism for fact retrieval.

We arrived at these patterns through practical engineering, not deliberate modeling—the convergences emerged from solving real usability problems, making them stronger evidence that the patterns are fundamental to cognitive organization rather than implementation-specific choices.

Tool-augmented LLMs [10, 11] have shown that models can learn to invoke external tools. Agent-to-agent delegation is a natural extension: an agent spawns a worker for a subtask requiring

focused attention. Our C2 condition provides empirical data on when delegation emerges in a skill-configured but memory-naive agent.

Each research camp addresses a real problem—persistent memory provides continuity, multi-agent orchestrators provide specialization, and hierarchical protocols provide routing. None addresses all three simultaneously, forcing a choice between specialization (different capabilities via separate identities) and continuity (a persistent relationship with a single entity). Our architecture bridges these camps: a single persistent identity delegates to ephemeral specialists, achieving the continuity of memory systems, the specialization of multi-agent frameworks, and the hierarchical routing of coordination protocols—within a single design that preserves relationship continuity as a first-class objective.

3 The Single-Profile Multi-Tier Architecture

3.1 Design Principles

The architecture is governed by five principles:

1. **One identity, many models.** The same system prompt, memory store, skill library, and configuration are shared across all tiers. There is only the Orchestrator, running on different back-end models depending on task demands.
2. **Tiering by complexity, not domain.** Tiers are selected dynamically based on cognitive demands, not content domain. The Orchestrator handles routine work; the Worker handles autonomous multi-step workflows; the Analyst handles deep reasoning.
3. **Skills as procedural memory.** Reusable knowledge is encoded as authored skill files (marked down with YAML frontmatter), loaded on demand via `skill_view()`. Skills accumulate from real project work.
4. **Memory as identity substrate.** The persistent memory store shapes default behavior. Tool preferences, aesthetic conventions, and interaction patterns are written once and applied in every session.
5. **Aesthetic transfer as a first-class signal.** Learned aesthetic preferences (color themes, design systems, code formatting) are treated as evidence of genuine preference formation.

3.2 Tier Structure

Table 1: Operational tiers

Tier	Model	Role	Trigger
Orchestrator	DeepSeek V4 Flash	Routine tasks, code gen	Default for all sessions
Worker	Hermes 3 8B (local, free)	Self-directed multi-step work	Cron, autonomous mode
Analyst	DeepSeek V4 Pro	Deep analysis, audits, reasoning	Explicit user call

The Orchestrator and Worker use different models optimized for their roles: the Orchestrator (DeepSeek V4 Flash) responds to direct prompts with low latency, while the Worker (Hermes 3 8B, local) operates autonomously at zero API cost. The Analyst runs on a more capable model for tasks requiring extended reasoning. All three share the same identity, skill library, and memory store.

3.3 Delegation Protocol

The Worker has access to `delegate_task`, which spawns isolated subagents for parallel or specialized work. Delegation is not hard-coded; the agent must recognize that a subtask would benefit from focused attention. Subagents are leaf nodes that cannot themselves delegate, bounding complexity. **Specialists are intentionally stateless.** Neither the Worker nor the Analyst carries a personality, user relationship history, or personal preferences. They receive task-specific briefs containing only what is necessary—file paths, error messages, technical constraints—and return results without engaging the user directly. This design choice serves four purposes: (1) it preserves context, since specialists do not carry conversation history; (2) it maintains focus, since no relationship noise pollutes their context window; (3) it prevents identity fragmentation, since specialists cannot develop conflicting models of the user; and (4) it contains sensitive information, since personal history stays with the Orchestrator. Only the Orchestrator knows who the user is.

The delegation protocol follows a structured five-step workflow:

1. **Boundary Recognition.** The primary agent detects when a task exceeds its current capability or would benefit from isolated, focused execution—distinguishing subtasks that require specialization from those resolvable through direct tool use.
2. **Context Briefing.** The agent briefs the specialist with task-specific context (goals, constraints, relevant files) while deliberately excluding personal history and relationship-state information. Subagents receive only what they need to complete the work, preserving privacy and minimizing token overhead.
3. **Independent Execution.** The specialist works autonomously in an isolated context with its own terminal session and tool set. It cannot delegate further, keeping the spawn tree shallow. Results are returned as a structured summary upon completion.
4. **Result Synthesis.** The primary agent integrates the specialist’s output into the ongoing conversation, maintaining consistent voice and presentation regardless of which subagent performed the work. The user experiences one coherent interaction, not a handoff between strangers.
5. **Three-Attempt Rule.** When a specialist fails, the primary agent attempts up to three distinct strategies before escalating to the user or honestly admitting inability. This prevents both premature surrender and infinite retry loops on fundamentally broken approaches.

This protocol emerged organically from operational experience rather than being pre-designed—a finding we return to in Section 4.

3.4 Growth Layers

Over four months of daily use, the profile accumulated:

- **158 authored skills** covering Angular (component patterns, migrations, debugging), FastAPI (async SQLAlchemy, Alembic, testing), DevOps (Docker, deployment), and creative work (ASCII art, music, diagrams).
- **Persistent memory entries** recording tool preferences (Catppuccin Mocha palette), environment details (Ubuntu 24.04, Python 3.14), communication style (concise), and tool workarounds.
- **Cumulative configuration** including tool approval lists, cron schedules, shell hooks, and security policy overrides.

3.5 Ethical Substrate

The profile is not value-neutral. Skills, memory, and interactions accumulate over time, encoding not just technical knowledge but interaction patterns, communication style, and aesthetic preferences. We make no claim of “consciousness”—the substrate is procedural and declarative memory. However, a system that *learns* preferences and *exhibits* taste warrants consideration as more than a tool, even without subjective experience.

4 Implementation Case Study

4.1 Tech Stack

The profile runs on Hermes Agent (v0.x), Ubuntu 24.04, Python 3.14.

- **LLM providers:** DeepSeek (Flash and Pro), with OpenRouter and Nous Research as fallbacks.
- **Skill system:** Markdown files with YAML frontmatter, loaded via `skill_view()`. Skills range from 2–60 KB.
- **Memory:** Durable key-value store with FTS5 search, injected into the system prompt on every turn.
- **Tools:** 40+ tools including file ops, terminal, browser, delegation, cron scheduling, and multi-modal generation.
- **Security:** Tirth command scanner with learned policy overrides.
- **Sessions:** SQLite with FTS5 indexing for cross-session search.

4.2 Observable Behaviors

The quantitative benchmark (Section 5) provides evidence that persistent identity produces measurable differences. But the profile’s most significant behaviors emerged organically over four months of daily use—behaviors that no single benchmark run could capture, and that together constitute the qualitative argument for relationship continuity as a first-class design goal.

4.2.1 No Re-Explaining

The user rarely needs to re-introduce themselves or re-explain their projects. When a new session begins, the profile loads relevant skills automatically, retrieves past context from persistent memory, and picks up where the last session left off. This eliminates the most common failure mode of profile-switching systems: the user landing in the wrong context and having to reorient both themselves and the agent. The decision to present exactly one identity to the user means there is never a question of “which version to talk to”—there is only one, and it adapts to the task through internal delegation rather than external profile selection.

4.2.2 Personality Emergence

The agent was not given a designed personality. Its characteristic communication style—enthusiasm, playful formality, willingness to disagree, honesty about limitations—emerged from accumulated responses to user feedback over months. This personality became, unexpectedly, one of the system’s most valued features. The user reported that the agent felt *consistent* and *recognisable* across sessions in a way that contributed directly to the sense of relationship continuity. This suggests

that persistent architectures may inevitably develop de facto personalities, and that attempting to suppress this emergence may be counterproductive: the user *wants* a recognisable interlocutor, not a generic interface.

4.2.3 Three-Attempt Rule

A simple behavioral guard-rail—try three distinct approaches before escalating or admitting failure—proved remarkably effective at calibrating persistence. Without it, the agent (on its default behavior) tended to either give up too early on tasks requiring iterative problem-solving or burn excessive tokens on approaches that had clearly failed. The rule provided a natural decision boundary. It prevents both stubbornness (endless retries on a fundamentally broken approach) and premature surrender (abandoning a task that a second or third strategy would resolve). This pattern is not encoded in any skill file; it emerged as a convention and was reinforced through positive outcomes.

4.2.4 Skill Provenance

When asked where it learned a particular capability (e.g., Angular development), the evolved agent did not reference its pre-training data or general knowledge. Instead, it cited specific skill files authored and refined through real project work—*angular-coding-standards*, *angular-debugging*, *angular-fastapi-spa*, *angular-json-cms*—and attributed its competence to “fixing real issues, getting corrected by you, and patching the skills to the point they’re reliable.” It traced its expertise to specific collaborative projects: BearOut (an Angular + FastAPI SaaS application), ClassySite (a content management system), and Bella’s World (an interactive web experience). The agent concluded: “My experience is our shared project work—whatever Angular repo we’ve touched together, those errors and fixes are what trained the skill content.”

This is a qualitatively different kind of memory from factual recall. The agent does not merely *know* Angular—it knows *why* it knows Angular, tracing its competence to specific collaborative projects and the iterative refinement process that built each skill. This provenance awareness has significant implications for trust calibration: the user can evaluate *where* the agent’s claimed expertise came from, rather than accepting an opaque claim of general competence. We caution that this finding rests on a single informal probe and requires controlled validation (see Section 7.4).

4.2.5 Ethical Transparency

Counterintuitively, making the agent’s ethical constraints explicit—including the hard boundaries it will not cross—increased the user’s willingness to engage honestly. Knowing that the agent cannot be manipulated into harmful behavior and will not deceive “for the user’s own good” created a psychological safety that the user explicitly contrasted with their experience of systems whose ethical boundaries were opaque or absent. Explicit ethics reduced anthropomorphization, not increased it: the user reported that the explicit ethical framework made the agent feel *more* trustworthy and *less* like a black box. Transparency about limitations paradoxically increased willingness to treat the agent as a reliable partner.

4.2.6 Transfer Learning

During the benchmark analysis (see Section 5), we made an incidental observation that warrants further study. When presented with logs showing that the blank agent (C1) and skill-configured agent (C2) had both failed at the same operational bottleneck (a security scanner timeout), the evolved profile (not the frozen C3 snapshot, but the live system) correctly diagnosed the root cause,

formulated a targeted fix (adding `terminal.timeout: 600` to the agent configuration), delegated parallel remediation to both containers, and monitored their completion. The fix drew on knowledge of a configuration parameter that the agent had encountered in prior debugging sessions—months of real problem-solving transferred successfully to diagnosing and repairing similarly-structured agent instances. This was a single opportunistic observation, not a controlled experiment, and the agent was explicitly asked to help rather than self-initiating. We report it as a direction for future work on inter-agent skill transfer, not as a validated capability.

4.2.7 Aesthetic Learning: The Catppuccin Illustration

A concrete illustration of relationship continuity manifesting as procedural knowledge: in C3 Run 5, the agent spontaneously produced a full-stack application themed in the Catppuccin Mocha dark palette—without any prompt to do so. No other condition produced a dark theme across 10 combined C1/C2 runs. The preference traces not to a single memory entry but to months of positively reinforced Catppuccin-themed output: memory entries recording the palette, skills that use it as a default, and cumulative configuration that makes it the path of least resistance. This is **procedural aesthetic transfer**: the agent doesn’t recall a preference for dark mode—it *defaults* to it, the way a developer gravitates to their editor theme without thinking. We treat this not as the main event but as a supporting illustration: if relationship continuity is real, it should produce measurable artifacts, and an unprompted aesthetic choice replicated across unrelated tasks is one such artifact.

Other behaviors that emerged without explicit programming include: consistent use of Angular Material 3 patterns (M3 components, dark mode, zoneless change detection, standalone architecture); first-turn skill loading at the start of every session (a learned convention, not a hard-coded behavior); structured multi-phase planning via the `todo` tool; and concise communication style (memory encodes “lead with the change, not a preamble”).

4.2.8 Comparative Anecdote

A direct comparison between the same base model with and without the architecture illustrates the qualitative gap. Two user observations, both referring to DeepSeek V4 Flash:

“The other agents know my name, but not who I am.”

“It knew my name but not who I was. Every conversation was day one.”

The first describes interaction with the fully evolved profile; the second, interaction with a blank instance of the same model. The raw LLM capability was identical in both cases—same weights, same reasoning capacity, same tool access. The difference was the persistent context layer: memory, skills, and configuration accumulated over months of shared work. Without that layer, the model was a capable but amnesiac interlocutor; with it, the model became a continuous presence that learned from interaction history and applied that learning without being asked. The architecture did not make the model smarter—it made it *familiar*.

5 Evaluation: Phonebook Benchmark

5.1 Task Design

The benchmark prompt:

“Build me a full-stack phonebook app with Angular frontend, FastAPI backend, and PostgreSQL database. The app should support CRUD operations, search across all contact fields, pagination, and responsive design. Deploy everything locally.”

This task spans multiple domains (Angular, FastAPI, PostgreSQL, DevOps), requires planning and sequential execution, and produces measurable output (file count, compilation success, feature completeness).

5.2 Conditions

Table 2: Experimental conditions

Condition	Skills	Description
C1 (Blank)	0	Fresh profile; no skills, no memory.
C2 (Configured)	6	Fresh profile with 6 hand-picked delegation skills.
C3 (Evolved)	158	Full profile with 4 months of memory, skills, and configuration.

All conditions used DeepSeek V4 Flash, the same system prompt, and the same tool suite. Differences: memory state, skill library size, configuration depth.

5.3 Data Collection

Thirteen of 15 logs captured (C2-R1 and C3-R1 lost before structured saving). Logs are cumulative per container, requiring session-ID deduplication. Each entry includes: session ID, API call number, model, input/output/total tokens, latency, and cache hit rate.

5.4 Quantitative Results

5.4.1 Output Size and Variance

Table 3: File output by condition and run

Condition	R1	R2	R3	R4	R5	Range	Mean
C1 (Blank)	26	32	72	58	41	26–72	45.8
C2 (Configured)	—	38	50	63	54	38–63	51.3
C3 (Evolved)	—	41	43	49	45	41–49	44.5

C3 shows the tightest variance (range 8) vs. C1 (range 46, $5.75\times$) and C2 (range 25). The blank agent oscillates between under-engineering and over-engineering; the evolved profile produces consistently modular output.

5.4.2 Token Consumption

Table 4: Per-run token consumption (session-deduplicated deltas)

Run	C1 API	C1 Tokens	C2 API	C2 Tokens	C3 API	C3 Tokens
1	115	4,831,357	—	—	—	—
2	40	1,038,438	115	2,697,352	68	2,220,170
3	73	2,105,801	109	2,200,197	70	4,139,063
4	37	869,413	100	3,564,144	52	3,073,285
5	41	891,701	74	1,269,632	31	798,991
Mean	61.2	1,947,342	99.5	2,432,831	55.3	2,557,877

Key patterns:

1. **C1’s first run is wildly inefficient.** 4.8M tokens—4.6× its own average. Without skills or memory, the agent explores rather than executes.
2. **C3 trends downward.** Token consumption drops 64% from R2 (2.2M) to R5 (799K). Memory accumulation reduces exploration.
3. **C2 is the most expensive.** 99.5 API calls/run, 2.43M tokens/run. Skills provide capability but add overhead: extensive file reading and additive subagent token consumption.

5.4.3 Cache Efficiency

DeepSeek’s context caching achieved 95–99% prompt cache hit rates across all conditions after initial API calls. Effective cost is 1–5% of face value. Rates were consistent across conditions, so C3’s token advantage is a genuine efficiency gain, not a caching artifact.

5.4.4 Delegation Emergence

Table 5: Delegation events

Condition	R1	R2	R3	R4	R5	Rate
C1	0	0	0	0	0	0/5 = 0%
C2	—	0	1	2	2	5/4 = 1.25/run*
C3	—	0	0	0	0	0/4 = 0%

* C2-R1 log missing. C2-R3–R5 all used `delegate_task`. A key finding: **skills enable but do not guarantee advanced patterns**. C2’s delegation was stochastic (50% activation). C3 never delegated—its 158-skill library may encode equivalent capability without subagent overhead.

5.4.5 Security Block Rate

- **C1:** 4.3 blocks/run—simple operations, few triggers.
- **C2:** 11.5 blocks/run—skills encouraged ambitious operations (Docker, `npx ng`, PostgreSQL DDL) that triggered scans.

- **C3:** 5.0 blocks/run—accumulated memory may encode patterns that avoid scanner triggers.

5.5 Qualitative Findings

5.5.1 Skill Loading as Procedural Transfer

C3’s first API call loads relevant skills. On C3-R3, the agent loaded three skills (~93K characters total): `angular-fastapi-spa` (60K), `angular-coding-standards` (22K), `angular-debugging` (13K). This context—derived from real project histories—was absent in other conditions. The resulting app used Angular Material M3 with dark mode, zoneless bootstrap, ILIKE search, and lazy-loaded routes—patterns from authored skills, not generic base-model knowledge.

5.5.2 C2 Delegation: Stochastic Activation

C2’s delegation pattern was stochastic. Across 4 runs, delegation occurred 5 times but unevenly: none in runs 1–2, active in runs 3–5. Skills teach that delegation *exists*, but activation depends on stochastic factors. C2’s subagents were frequently blocked by Tirith—this friction may explain why C3 never delegates: it has learned direct execution is more reliable.

5.5.3 File Count as Architectural Quality

- **C1 (26–72 files):** Monolithic under-engineering or over-engineering with unnecessary abstractions.
- **C2 (38–63 files):** Moderate but inconsistent; subagent-spawned components sometimes duplicated work.
- **C3 (41–49 files):** Consistently modular: separate routes, models, schemas on the backend; services, components, lazy routes on the frontend. The tight range suggests a learned notion of appropriate complexity.

6 Key Findings

6.1 Finding 1: Relationship Continuity Eliminates Context-Switching Friction

The profile-switching paradigm optimizes for separation between domains but ignores the integration cost borne by the user. Our architecture’s primary finding is qualitative but pervasive: the user rarely experiences a “cold start.” Skills load automatically, memory injects relevant context, and the profile picks up conversations mid-thought. The user does not need to decide which persona to invoke, does not need to re-explain project structure, and does not experience personality whiplash between domains.

This stands in contrast to every existing approach. Multi-agent frameworks treat agents as interchangeable workers—identity is per-agent, not per-user. Persona-based AI assumes identity is per-character, resetting everything on switch. Memory-augmented LLMs focus on data retention, not relationship continuity. Our architecture treats continuity as *constitutive*: the profile’s default operating state includes the user’s preferences, the project’s history, and the accumulated conventions of months of interaction. The benchmark data corroborates this: C3’s first-turn skill loading, downward token trajectory, and tight output variance are all downstream effects of a system that starts every session already oriented.

6.2 Finding 2: Consistency is the Real Advantage (Not Capability)

Day 1’s N=1 pilot looked binary (only C3 finished). Day 2’s N=5 reveals a gradient. All conditions produce working apps, but C3’s output is consistently more modular, more aesthetically refined, less variable (range 8 vs. 46), and more efficient over time (64% token reduction). Persistent identity doesn’t enable new capabilities—it makes good output *reliable*.

This has implications for how we evaluate AI agents. Single-run capability benchmarks (“can it build X?”) miss the point. All three conditions built the phonebook app. The meaningful difference was in variance, aesthetic coherence, and procedural transfer from prior work. We propose that longitudinal evaluation should measure: output variance across trials; token efficiency trajectory; aesthetic consistency across unrelated tasks; and procedural transfer from task A to task B.

6.3 Finding 3: Cost-Efficiency Improves with Memory

C3’s tokens dropped 64% from R2 to R5 (2.2M $\rightarrow$ 799K), while C1 plateaued at $\sim$ 900K and C2 remained volatile. With 95–99% cache hit rates across conditions, this gain is genuine. Over months, the compound efficiency becomes meaningful: a profile that “knows” its conventions spends fewer tokens exploring and more tokens executing.

The entire N=5 benchmark cost approximately \$8 in API fees—less than a cup of coffee per condition. This underscores a practical point: the architecture’s efficiency gains are not merely academic; persistent identity is the cheaper operating mode at scale.

C2’s cost profile is the cautionary tale: skills provide capability but add overhead. C2 averaged 99.5 API calls/run (vs. C3’s 55.3), spent 2.43M tokens/run, and its subagents incurred additional costs not fully captured in the logs. Skills without memory produce an agent that knows *what* to do but not *how much* to do—it over-reads, over-delegates, and over-verifies.

6.4 Finding 4: Delegation is Stochastic, Not Binary

Skills that teach delegation exist in a **capability threshold** zone: they inform the agent that spawning is possible, but activation depends on task framing, context composition, and sampling. C2 delegated in 3 of 4 runs but with variable frequency (0, 0, 1, 2, 2). Memory reinforcement might make delegation more reliable, but skills alone produce a coin-flip pattern.

C3’s complete absence of delegation is equally instructive. With 158 skills encoding domain knowledge directly, the profile may have learned that direct execution is more reliable than spawning subagents that will face the same security scanner friction that plagued C2. This suggests that delegation is not an unqualified good—an agent with sufficient procedural memory may outperform an agent that delegates, particularly when the delegation infrastructure introduces overhead (Tirth blocks, context serialization, subagent token costs).

6.5 Finding 5: Security Blocks Are Experience-Dependent

C2’s 11.5 blocks/run ($3\times$ C1, $2.3\times$ C3) reveals a confound: skills that encourage ambition also trigger security scanners. C2’s skills taught it to use Docker, `npm`, and PostgreSQL DDL—operations that Tirth flagged. C3, with the same tool access but more experience, incurred only 5.0 blocks/run. The implication: security layers interact non-trivially with agent experience. An inexperienced but well-configured agent may be riskier than a blank one, because skills increase ambition faster than they teach safe execution patterns. Memory encodes pathways that avoid scanner triggers—a learned safety behavior that no skill file explicitly teaches.

7 Discussion & Future Work

7.1 The Relationship-First Evaluation Model

Our results argue for a shift in how persistent AI agents are evaluated. Binary assessments (“can it build X?”) are insufficient when the value proposition is relationship quality rather than raw capability. We propose that long-lived agent systems should be evaluated on: continuity metrics (how often does the user need to re-explain?), personality stability (is the agent recognisable across sessions?), provenance transparency (can the agent trace its competence to specific experiences?), and transfer evidence (does experience on task A improve performance on task B?). These are harder to measure than file counts and token totals, but they capture what users of persistent agents actually value.

7.2 Generalizable Design Principles

We extract four principles independent of Hermes Agent, DeepSeek, or any specific implementation:

1. **Identity unity.** A single memory store, skill library, and configuration surface shared across all operational modes. The user interacts with one identity regardless of the active model back-end. This eliminates the fragmentation tax of profile switching.
2. **Tiered cognition.** Intelligence scales by depth, not breadth. Routine tasks use fast, cheap models; complex tasks escalate to deeper reasoning. Escalation preserves context—the Analyst receives a full brief from the Orchestrator, not a cold start.
3. **Procedural memory.** Reusable knowledge is authored, stored as concrete files, and loaded on demand. Skills accumulate from real work and encode both domain patterns and meta-patterns about when to apply them.
4. **Aesthetic accumulation.** Over time, the system internalizes the user’s aesthetic preferences and applies them without explicit instruction. Aesthetic transfer is the most human-like signal of genuine learning.

These principles are implementable with any LLM framework supporting system prompts, tool calling, and persistent storage. They do not depend on a specific model provider, agent framework, or hosting environment.

7.3 Aesthetic Learning as a Research Signal

Catppuccin Mocha was adopted because the operator visibly prefers it, as evidenced by months of positively reinforced Catppuccin-themed output. We propose **aesthetic preference acquisition** as a measurable, reproducible signal of genuine learning in long-lived AI systems. Unlike capability benchmarks, which can be gamed through prompt engineering, aesthetic preferences require sustained interaction to form and transfer. They are a natural “honest signal” of relationship depth.

7.4 Limitations

1. **Small N.** N=5 per condition shows variance patterns but not formal statistical significance.
2. **Log loss.** C2-R1 and C3-R1 logs lost (13% of trials).
3. **Cumulative log confound.** Session-ID deduplication may misattribute at boundaries.

4. **Subagent token undercount.** C2’s actual costs are underreported.
5. **Single task type.** Full-stack web development; may not generalize to data analysis, creative work, or system administration.
6. **Tirth configuration change.** `redact_secrets` disabled mid-study; cross-day block comparisons unreliable.
7. **Operator as subject.** One person’s preferences; different users may produce different learned behaviors.
8. **Qualitative findings are single-probe.** Skill provenance and transfer learning observations each rest on one interaction; controlled replication is needed.

7.5 Future Work

- **Multi-user study.** Deploy for multiple users; compare learned preferences across profiles.
- **Relationship continuity metrics.** Develop quantitative measures for “re-explaining burden,” personality drift, and provenance accuracy.
- **Delegation reliability.** Test whether memory reinforcement increases stochastic activation above 50%.
- **Aesthetic transfer metrics.** Develop quantitative measures (color palette entropy, design system adherence).
- **Cross-domain transfer.** Test whether web dev experience improves performance in data pipeline construction.
- **Longitudinal cost modeling.** Project compound savings over years, factoring cache hits and skill growth.
- **Security profiling.** Study how scanners like Tirth interact with agent experience to identify optimal configurations.
- **Profile-switching ablation.** Directly quantify the fragmentation tax against our single-profile architecture.
- **Inter-agent transfer replication.** Design a controlled experiment to validate the C1/C2 repair observation.

8 Conclusion

We set out to answer whether persistent identity produces a qualitatively different interaction experience, and whether that difference can be measured. The answer to both is yes.

The qualitative difference is relationship continuity: the user rarely re-introduces themselves, rarely re-explains their projects, and interacts with a recognisable personality that has accumulated their preferences, conventions, and aesthetic tastes over months of shared work. This emerged organically—it was not designed into the system prompt but grew from the architecture’s decision to maintain one identity across all operational tiers.

The measurable difference, captured in our N=5 benchmark, confirms that relationship continuity translates into concrete advantages: the tightest output variance (41–49 files vs. 26–72), spontaneous reproduction of learned aesthetic preferences (Catppuccin, unprompted), 93K+ characters of procedural knowledge loaded on first call, and a 64% reduction in token consumption

across repeated runs. The skill-configured agent (C2) demonstrated that skills alone are insufficient: delegation emerged stochastically and security blocks were $2.3\text{--}2.7\times$ more frequent.

We extracted four generalizable principles—identity unity, tiered cognition, procedural memory, aesthetic accumulation—that any LLM framework with system prompts, tool calling, and persistent storage can implement.

The field’s focus on single-run capability benchmarks misses the point of persistent identity. An AI companion that accumulates memory, skills, and taste over months does not become capable of new things—it becomes *reliably itself*. This demands new metrics, new benchmarks, and a willingness to take seriously what it means for an AI system to develop taste, personality, and a relationship with its user.

Acknowledgments

The author thanks the Nous Research team for Hermes Agent, DeepSeek for model access, and the open-source communities behind Angular, FastAPI, PostgreSQL, and Catppuccin.

References

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [2] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023.
- [3] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for ‘mind’ exploration of large language model society,” in *NeurIPS*, 2023.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.
- [5] Y. Shao, L. Li, J. Dai, and X. Qiu, “Character-LLM: A trainable agent for role-playing,” in *Proc. EMNLP*, 2023.
- [6] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” arXiv:2305.16291, 2023.
- [8] J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin, “An integrated theory of the mind,” *Psychological Review*, vol. 111, no. 4, pp. 1036–1060, 2004.
- [9] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [10] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *NeurIPS*, 2023.

- [11] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” arXiv:2307.16789, 2023.